

ADR-002 — 주문 retry, partial fill과 reconciliation

항목	값
상태	Accepted
결정일	2026-08-20
최종 검토일	2026-08-29
적용 결정	D-08
적용 상품	Binance Spot long-only 주문

1. 배경

주문 제출 timeout은 거래소가 주문을 받지 않았다는 뜻이 아니다. **NEW**, **PARTIALLY_FILLED**, 응답 유실과 로컬 저장 실패를 일반 실패처럼 재제출하면 중복 주문이나 포지션 불일치가 생긴다. 현재 TradingSTM은 같은 주문 ID 우선 조회, **STOPPING**, **RECONCILIATION_REQUIRED**와 구체 결과 event의 경계를 이미 갖고 있지만, 횡수·간격·잔여 수량·재시작 정책은 확정되지 않았다.

2. 공통 용어와 불변식

- **intent_id**: 한 번의 전략 결정 또는 강제 청산 의도를 식별한다.
- **client_order_id**: application session UUID, **intent_id**와 제출 attempt를 결합한 거래소 idempotency key다. 같은 session·intent·attempt에서는 결정론적으로 같고, 새 session에서는 완료된 Binance ID를 재사용하지 않는다.
- **exchange_order_id**: 거래소가 주문을 수락한 뒤 부여한 ID다.
- **submission_attempt**: 실제 신규 주문 제출 횡수다.
- **reconciliation_attempt**: 같은 주문을 조회하거나 취소 결과를 확인한 횡수다.

다음 불변식은 모든 branch에 적용한다.

1. 같은 **client_order_id**의 제출 결과가 불명확하면 새 ID로 주문하지 않는다.
2. **exchange_order_id**를 받지 못했어도 **client_order_id**로 먼저 조회한다.
3. fill은 (**exchange_order_id**, **trade_id**)로 deduplicate하고 새로 확인된 delta만 Position에 반영한다. **Order**는 이미 반영한 fill ID를 보존한다.
4. 실제 fill은 주문 상태보다 우선한다. **CANCELED**나 **EXPIRED**에도 fill이 있으면 Position·Trade·Performance에 먼저 반영한다.
5. 성공 결과 event는 Position 반영과 거래 이력의 durable 저장이 모두 성공한 뒤에만 TradingSTM에 전달한다.
6. **UNKNOWN**, active 주문, 저장 실패에서는 같은 거래 의도를 새로 제출하지 않는다.
7. retry와 reconciliation은 Controller scheduler가 수행하며 재귀 호출이나 busy loop를 사용하지 않는다.
8. Matching Engine의 **-2010**은 **Duplicate order sent.**를 포함할 수 있으므로 일반 4xx rejection으로 terminal 확정하지 않고 **UNKNOWN**으로 같은 ID만 조회한다.
9. Production 조립은 process당 하나의 interruptible event runtime worker만 사용한다. worker는 **TradingController**의 bounded runtime cycle을 깨울 뿐이며, cycle이 **RETRY_BACKOFF** due 작업과 직렬 event queue를 순서대로 처리한다. route별 drain, retry별 timer/thread와 busy loop는 사용하지 않는다.

10. bounded cycle 또는 terminal 상태 publication이 실패하면 worker는 재시작하지 않고 해당 process의 event-runtime gate를 **RECONCILIATION_REQUIRED**로 영구 잠근다. queue, pending journal과 same-ID 주문 사실은 보존하고 새 주문과 안전 종료는 차단한다.
11. 제출 전 pending-order **PREPARED** UPSERT가 예외를 반환하면 REST POST는 보내지 않지만, file 또는 parent-directory fsync가 이미 끝난 뒤 예외가 발생했는지는 호출자가 판별할 수 없다. 이 모호한 durability cut-point에서는 청산 세션을 **NOT_STARTED**로 rollback하거나 동일 intent를 다시 만들지 않고 **RECONCILIATION_REQUIRED**와 operator lock을 유지한다. sidecar가 실제로 존재하면 same client ID 복구 근거로 보존하고, 존재하지 않아도 현재 process에서는 비어 있다고 추측해 command gate를 다시 열지 않는다. 다음 process는 journal을 성공 replay한 뒤 아래 §6.2의 schema별 제출 경계 근거를 적용한다. 현재 process의 모호한 save 반환을 restart 정책으로 업그레이드하지 않는다.
12. 복구 Position의 최초 force-sell은 free balance와 exchange filter 적용 뒤에도 requested/submitted 수량이 authoritative Position 전량과 정확히 같을 때만 **PREPARED**를 저장한다. max-notional의 승인 의미는 노출을 늘리는 BUY entry cap이다. 일반 SELL의 기존 local cap은 유지하되, 가격 상승 뒤 노출을 줄이는 STOP/recovery SELL에는 적용하지 않는다. filter 내림으로 부분 청산을 시작하지 않는다.
13. 현재 process가 모르는 **bat-** execution report는 callback 실패로 조용히 버리지 않는다. 즉시 **RECONCILIATION_REQUIRED**를 게시하고 account recovery worker를 깨워 open/recent와 account snapshot barrier를 다시 수행한다.

3. 조회와 제출 retry 예산

3.1 같은 주문 조회

NEW, **PARTIALLY_FILLED**, network timeout, HTTP 응답 유실 또는 **UNKNOWN**이면 같은 주문을 최대 4회 조회한다.

조회 attempt	기본 대기
1	1초
2	2초
3	4초
4	8초

- 각 기본 대기에 -20% 이상 $+20\%$ 이하의 jitter를 적용한다.
- 테스트에서는 주입한 jitter factor **1.0**으로 정확히 **1, 2, 4, 8초**를 검증한다.
- **429/418** rate-limit 응답에 유효한 **Retry-After**가 있으면 그 wait-not-before를 축소하지 않고 우선한다. 표현할 수 없는 값은 짧은 대기로 대체하지 않고 fail closed한다. 이 요청도 attempt를 소비한다. Binance가 명시한 시각 전에는 같은 주문 조회도 수행하지 않는다.
- 4회 뒤에도 거래소 사실을 확정할 수 없으면 **RECONCILIATION_REQUIRED**로 전이하고 신규 전략 action과 자동 종료를 차단한다.

3.2 terminal zero-fill 뒤 신규 제출

기존 주문이 **REJECTED**, **CANCELED**, **EXPIRED** 또는 동등한 terminal 상태이고 누적 fill이 0임을 조회로 확인한 경우에만 같은 intent의 새 attempt를 허용한다.

- 최초 제출 뒤 최대 4회의 신규 제출 retry를 허용한다. 총 제출 상한은 5회다.
- 일반 진입-청산 retry는 **1, 2, 4, 8초**의 bounded exponential schedule과 같은 $\pm 20\%$ jitter를 사용한다.

- UI Rule ER-10의 강제 매도 정책은 더 구체적인 기존 규칙이므로 force-sell 신규 제출은 고정 3초 간격, 최대 4회 retry를 사용한다.
- 각 retry는 새 `client_order_id`를 쓰되 동일 `intent_id`와 증가한 attempt 번호를 포함한다. 이전 주문이 terminal임을 먼저 증명해야 한다.
- 예산 소진 시 `*_BUY_FAILED`, `*_SELL_FAILED` 또는 `FORCE_SELL_FAILED`를 전달하고 Position이 남아 있으면 `RECONCILIATION_REQUIRED`를 유지한다.

Worker의 interruptible 주기 wake는 due 시각을 앞당기는 정책이 아니다. Controller가 주입 clock으로 정확한 due 여부를 다시 검사하므로 same-ID 1·2·4·8초와 force-sell 고정 3초는 그대로 유지되고, wake 지연만 worker poll 상한 안에서 발생할 수 있다.

4. 상태별 처리 표

관찰 결과	Controller 처리	신규 제출 허용	완료 event
최초 응답 FILLED	fill dedup-집계 → Position → history 저장	아니오	실제 side/strategy의 성공 event
NEW	같은 ID 조회 schedule	아니오	없음
PARTIALLY_FILLED + active	새 fill delta만 Position/runtime에 선반영하고 같은 ID 조회. final Trade는 아직 만들지 않음	아니오	없음
UNKNOWN 또는 timeout	exchange ID 또는 client ID로 같은 주문 조회	아니오	없음
terminal + zero fill	pending ID 해제 후 retry 예산 평가	예, 예산 안에서만	실패 event 또는 다음 attempt 결과
terminal + partial BUY	실제 fill을 최종 진입량으로 확정하고 자동 top-up 금지	아니오	CASE_*_POSITION_OPENED
terminal + partial SELL	실제 fill 반영, owner와 동일 exit intent 유지, 잔여량만 retry	예, 이전 주문 terminal 확인 후	수량 0일 때만 CASE_*_SELL_FILLED
terminal + partial force-sell	실제 fill 반영, 잔여 Position만 3초 정책으로 retry	예, 이전 주문 terminal 확인 후	수량 0일 때만 FORCE_SELL_FINISHED
복구 force-sell prepare가 잔량 전량을 보존하지 못함	최초 effect 전이면 NOT_STARTED 원자 복원, terminal partial 뒤면 operator reconciliation	아니오	없음
조회 4회 뒤 불명	운영자 개입이 필요한 lock 상태	아니오	없음

관찰 결과	Controller 처리	신규 제출 허용	완료 event
Position 반영 실패	거래소 사실 보존, session lock 및 재구성	아니오	없음
history append/fsync 실패	같은 order/fill의 저장만 재시도	아니오	없음

부분 BUY를 목표 수량까지 자동 보충하지 않는 이유는 이미 포지션이 열린 상태에서 동일 진입 판단을 다시 실행하면 의도보다 큰 포지션이 될 수 있기 때문이다. 부분 SELL은 기존 청산 의도가 이미 확정되어 있으므로 실제 잔여 Position만 계속 정리한다.

TradeHistory의 한 record는 한 terminal order의 누적 fill을 집계한 ExecutionSummary다. active partial 상태에서는 final Trade를 append하지 않는다. terminal이 확인되면 그 주문의 모든 fill로 한 Trade를 저장한다. process가 그 전에 종료되면 startup reconciliation이 거래소 fill을 다시 조회하고 fill ID dedup으로 Position을 재구성한다. 이 규칙은 같은 order ID에 내용이 달라지는 JSONL record를 여러 번 쓰는 일을 막는다.

5. STOP 중 pending 주문

STOP_CONFIRMED가 pending 주문과 겹치면 다음 순서를 지킨다.

1. TradingSTM의 G-06P로 STOPPING에 진입해 신규 전략 action을 차단한다.
2. queryOrderResult(...)로 현재 상태와 fill을 먼저 확인한다.
3. 주문이 active이면 cancelOrder(...)를 한 번 요청한다.
4. cancel 응답만 믿지 않고 같은 주문을 다시 조회해 terminal 상태와 fill을 확인한다.
5. 확인한 fill을 Position과 history에 idempotent하게 반영한다.
6. 잔여 Position.quantity > 0이면 force-sell을 수행한다.
7. 잔여 수량이 0이고 pending 주문이 없을 때만 FORCE_SELL_FINISHED를 전달한다.

cancel timeout 또는 상태 불명 상태에서는 force-sell을 동시에 제출하지 않는다. 기존 매수 주문이 뒤늦게 체결되는 동안 전량 매도를 제출하면 naked sell 또는 잔여 포지션이 생길 수 있으므로 RECONCILIATION_REQUIRED에서 사실 확인을 계속한다.

6. 저장 실패와 재시작 복구

6.1 저장 실패

- 거래소 체결과 Position 반영은 되돌리지 않는다.
- TradeHistoryRepository.saveThisTradeByOrderID(...)를 동일 order ID로 재시도한다.
- 메모리 TradeHistory와 Performance는 dirty로 표시하고, durable history와 확인한 execution을 이용해 재구성할 때까지 신규 주문을 차단한다.
- 저장 성공 전에 성공 결과 event를 보내지 않는다.
- 저장 실패 때문에 원 주문을 다시 제출하지 않는다.

6.2 process restart

backend startup은 trading start를 받기 전에 다음 reconciliation을 완료해야 한다.

1. JSONL 이력과 마지막 정상 order ID를 읽는다.

2. 외부 POST 전에 file과 parent directory까지 fsync한 pending-order sidecar의 **PREPARED** 항목을 읽는다. legacy sidecar v1/v2의 UPSERT도 **PREPARED**로 해석한다.
3. **APIGateway.listOpenOrderResults(symbol)**로 앱이 생성한 open order를 조회한다.
4. 앱의 client order ID prefix에 해당하는 최근 execution을 조회해 이력 이후 fill을 찾는다.
5. 각 pending 주문은 신규 submit 없이 **1·2·4·8초** 뒤 같은 ID로 조회하고 누락 fill을 Position/History에 idempotent하게 반영한다. 단, Testnet reset이 과거 숫자 **orderId**를 다른 client ID에 재사용한 충돌은 Position 반영과 sidecar 삭제 전에 차단한다.
6. Schema v3 writer는 **PREPARED** UPSERT를 fsync한 뒤 REST POST 직전에 **SUBMITTED** transition을 file.parent-directory까지 fsync하고, 이 transition이 실패하면 POST를 시작하지 않는다. 따라서 replay된 v3 **PREPARED**는 POST 미시작 근거이며, 같은 ID가 **1·2·4·8초** 조회에서 모두 정확히 **-2013**이고 transport/**UNKNOWN**이 없을 때만 sidecar **REMOVE**를 fsync해 active lock을 해제한다. 최초 UPSERT의 intent/attempt/schema와 후속 REMOVE line은 append-only audit에 남는다.
7. Legacy v1/v2 **PREPARED**는 거래소가 POST를 수락한 직후 응답 전에 process가 종료된 상태를 배제할 수 없다. 따라서 네 번 모두 **-2013**이어도 미제출로 추측하거나 sidecar를 삭제하지 않고 **RECONCILIATION_REQUIRED**를 유지한다. 동일 ID의 legacy UPSERT 뒤 v3 UPSERT를 덧붙여 provenance를 세탁하는 journal은 손상으로 거부한다.
8. 최초 submit이 Matching Engine 이전의 typed rejection으로 확정되면 lifecycle sidecar를 **SUBMISSION_REJECTED_CONFIRMED**로 전이하고 file과 parent directory를 fsync한 뒤에만 메모리 상태를 확정한다. 이 상태에서만 같은 ID 조회가 네 번 모두 정확한 **-2013**이고 transport/**UNKNOWN** 관찰이 한 번도 없을 때 미제출을 확정해 sidecar를 제거한다.
9. history commit 뒤 sidecar REMOVE만 실패한 항목은 같은 client ID를 다시 조회해 terminal status, (**clientId**, **orderId**) pair, metadata와 누적 execution summary가 durable Trade 한 건과 정확히 같을 때만 sidecar를 제거한다. 누적 summary는 수량·금액, 평균가, fee asset·원 금액·quote 환산액과 마지막 fill 시각을 포함한다.
10. history 저장과 sidecar REMOVE 내구성은 서로 다른 marker로 추적한다. reconnect는 실제 sidecar를 다시 읽고 REMOVE fsync가 끝나지 않았으면 signed stream을 유지하더라도 외부 command gate를 열지 않는다.
11. open order와 Position이 모두 설명되면 application command barrier를 READY로 열되, 일반 trading session은 **NOT_STARTED**로 유지한다. Position이 0이면 이후 명시 start를, Position이 양수면 ADR-003의 명시적 recovered-position liquidation만 허용한다. 자동 resume나 startup 중 전략 event 전달은 금지한다.
12. 설명할 수 없는 주문·fill·잔액 차이가 있으면 **RECONCILIATION_REQUIRED**를 유지하고 운영자에게 order ID와 차이를 표시한다.

복구가 끝나기 전에는 market event를 STM에 전달할 수 있어도 외부 주문 Action은 실행하지 않는다.

6.3 account stream 재연결 barrier

account user-data stream이 비정상 종료되면 transport 연결 여부와 별개로 신규 주문 gate를 즉시 닫고 다음 순서를 한 번의 reconciliation barrier로 수행한다.

1. 첫 full account REST snapshot을 적용하고 local overlay를 비운다.
2. 새 signed user-data stream의 subscribe ACK를 받은 뒤에만 exchange open/recent 주문과 모든 local unresolved 주문의 same-ID 결과를 조회한다.
3. 설명되지 않은 app-prefix open/fill, durable BUY provenance 누락과 **Position.quantity > account base balance**를 차단한다. numeric order ID collision은 same-ID result를 Order/Position에 적용하기 전 검사하고, terminal fill은 durable (**clientId**, **exchange ID**)와 execution summary가 정확히 같아야 한다.
4. ACK 이후의 두 번째 full account REST snapshot을 적용해 구독 전 account gap을 닫고 Position과 balance를 다시 비교한다.

- 수신 loop는 account event를 bounded 단일 FIFO dispatcher에 넣는다. enqueue 순간부터 callback 완료까지 `account_ready = false`이며, barrier 중 event가 하나라도 대기하면 현재 구독을 닫고 reconciliation을 다시 수행한다.
- queue overflow, consumer/worker failure, disconnect, REST/rebase 검증 실패는 모두 새 subscription을 닫고 `RECONCILIATION_REQUIRED`를 유지한다. 조용한 barrier가 완결되고 현재 subscription이 connected-caught-up일 때만 command gate를 다시 연다.
- gate commit 직후 optional application observer를 같은 Controller/application RLock에서 호출해 authoritative Account와 열린 trading lifecycle을 순서대로 게시한다. observer 실패는 event runtime failure로 영구 fail closed해 backend-only 주문 재개를 금지한다.

이 순서는 마지막 주문 REST 조회 전에 stream을 먼저 여므로 subscribe ACK 직전의 체결은 REST에서, ACK 이후의 체결은 stream backlog 또는 REST 결과에서 관찰된다. callback을 application lock 아래 억지로 실행하거나 backlog가 남은 상태를 ready로 추측하지 않는다.

7. 기존 클래스 Operation 결정

새 업무 클래스를 만들지 않는다. 거래소 주문 사실을 소유한 기존 `APIGateway`에 다음 Operation을 배치한다.

```
queryOrderResult(
    symbol : String,
    orderId : Long? = null,
    clientOrderId : String? = null
) : OrderResult

cancelOrder(
    symbol : String,
    orderId : Long? = null,
    clientOrderId : String? = null
) : OrderResult

listOpenOrderResults(symbol : String) : List<OrderResult>

listRecentOrderResults(
    symbol : String,
    limit : int = 100
) : List<OrderResult>
```

조회와 취소는 `orderId` 또는 `clientOrderId` 중 정확히 하나 이상을 요구한다. 전략 Guard, retry 예산과 Action 순서는 `TradingController`가 소유하고 Gateway는 Binance 응답을 정규화하는 책임만 가진다.

8. 검증 의무

- ☑ timeout, unknown, active, partial, terminal 상태의 처리가 확정되었다.
 - ☑ 조회와 신규 제출의 횟수·간격이 구분되어 확정되었다.
 - ☑ pending STOP의 query → cancel → query → fill → 잔여 매도 순서가 확정되었다.
 - ☑ 저장 실패와 재시작 복구에서 중복 주문 금지가 확정되었다.
 - ☑ Phase 8에서 상태별 fault matrix와 deterministic scheduler test를 구현했다.
- `test_order_fault_matrix.py`, `test_order_reconciliation_flow.py`,

`test_order_observed_time_scheduling.py`에서 동일 주문 조회, 1·2·4·8초 일정, terminal zero-fill 확인, 총 5회 제출 상한과 force-sell 3초 재시도를 결정론적으로 검증한다.

- ☒ Phase 9에서 accepted-response timeout, 누적 partial fill, duplicate와 disconnect를 in-memory transport로 결정론적으로 주입한 opt-in suite 2개를 통과했다. 이 검증은 외부 Binance credential이나 network 실행 증거가 아니다.
- ☒ Phase 9에서 pending sidecar, same-ID startup query, history-commit/REMOVE crash, confirmed-rejection 전이, session별 client ID 고유성, subscribe-ACK 이후 주문 snapshot과 두 account REST snapshot, 설명되지 않은 최근 fill, dispatcher barrier와 Testnet reset provenance를 local integration test로 검증했다.
- ☒ Phase 9 recovered-position liquidation에서 **PREPARED** UPSERT가 file-directory fsync 뒤 예외를 반환하는 fault를 주입해 REST POST 0회, durable journal 보존, session rollback 금지, **RECONCILIATION_REQUIRED**와 command gate 폐쇄를 검증했다.
- ☒ Phase 13에서 v3 **PREPARED**의 SUBMITTED-fsync-before-POST provenance를 typed replay하고, 4회 exact absence 뒤 신규 POST 없이 REMOVE-gate 복구·attempt audit 보존을 검증했다. v1/v2 **PREPARED**는 계속 fail closed하고 legacy→v3 provenance 세탁은 손상으로 차단한다.
- ☒ Phase 9에서 복구 Position의 free-filter 전량 preflight와 terminal partial 뒤 residual filter 실패의 operator lock, 알 수 없는 **bat-** execution report의 recovery wake와 authoritative lifecycle publication을 검증했다.
- ☒ 실제 Testnet credential와 주문 opt-in 0으로 account/Kline/open/recent/commission/ signed stream read-only parity를 통과한다.
- ☒ 사용자 승인 BUY entry cap 10 **USDT**에서 current **ETHUSDT LOT_SIZE**, **MARKET_LOT_SIZE**, **NOTIONAL**을 mutation 전에 조회하고 유효한 0.0040 **ETH** MARKET BUY만 허용했다. actual BUY/force-sell lifecycle과 process A 종료 뒤 fresh recovery SELL/fresh Position 0 replay를 통과했으며 최종 pending과 matching open order는 0건이다.